ASPIRE: Agentic /Skills Discovery for Robotics

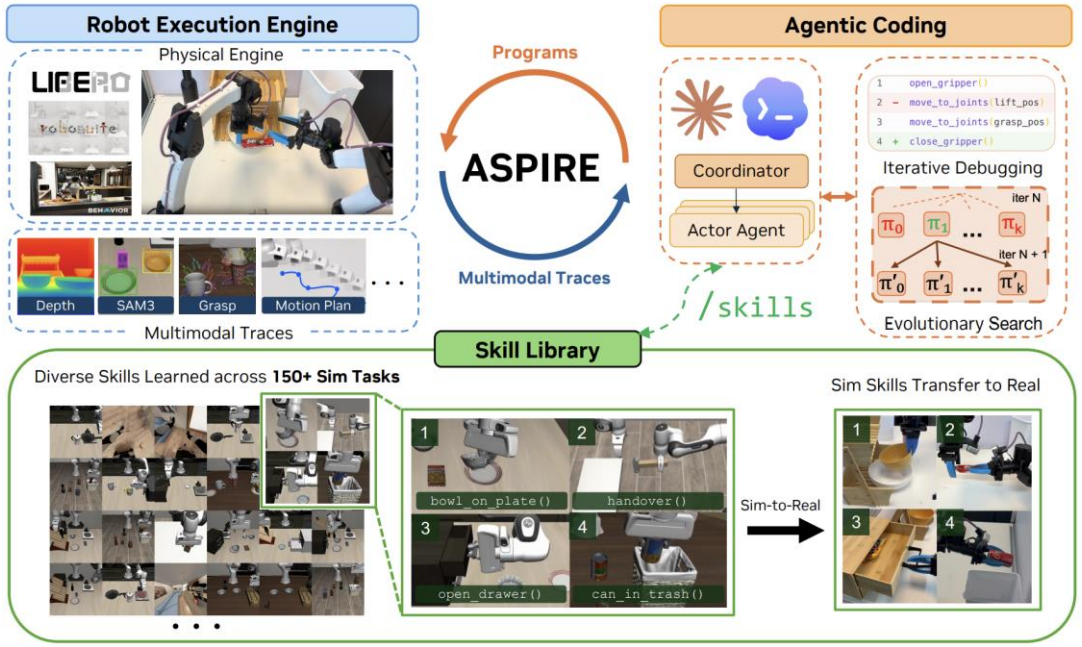
2026. 09. 16

Seonyeong Jeong

Covered Paper

- ASPIRE: Agentic /Skills Discovery for Robotics (NVIDIA 2026)

  ➡ Code-as-Policy Continual Learning



Preliminaries

Code-as-Policy란?

에이전트가 주어진 상황에서 어떤 행동을 할지 결정하는 함수를 '정책(policy)'이라 부름

Code-as-Policy는 이 정책을 사람이 읽고 수정할 수 있는 실행 가능한 프로그램 코드로 표현하는 방법론

LLM이 미리 정의된 인식·제어 API를 조합해 코드를 작성하면, 그 코드 자체가 에이전트의 행동 정책이 됨

Continual Learning이란?

하나의 에이전트가 여러 작업을 순차적으로 경험하면서, 이전에 학습한 내용을 잊지 않고 새로운 지식을 계속 누적해 나가는 학습 패러다임

배포 이후에도 경험이 쌓일수록 점점 더 나은 성능을 내는 것이 목표이며, 기존 지식을 잃어버리는 현상(catastrophic forgetting)을 막는 것이 핵심 과제

Evolutionary Search(진화적 탐색)란?

생물의 진화 과정에서 착안한 최적화 기법

여러 후보를 동시에 생성해 평가한 뒤, 성능이 좋은 후보를 선택 및 변형해 다음 세대를 만드는 과정을 반복함

하나의 정답을 조금씩 고쳐나가는 방식보다, 서로 다른 여러 대안을 넓게 탐색할 수 있다는 장점이 있음

Introduction

기존 Robotic coding agents의 문제점

- Coarse task-level feedback만 제공
- Debugging 어려움

  : 다양한 상호작용하는 요소들에서 failure이 발생할 수 있기 때문

  → Fine-grained diagnostic traces 없이는 어떤 증거를 검사할 지, 어떻게 실패를 localize할 지, 어떤 repair 전략을 시도할지 결정 불가
- 다양한 task들에서 경험을 누적해서 모으지 않음

  = task 끝나면 recovery 전략 등을 discard함

  △ 인간 로봇 프로그래머들은 경험을 통해서 reusable한 knowledge 쌓음 → 시간이 지날수록 더 효과적

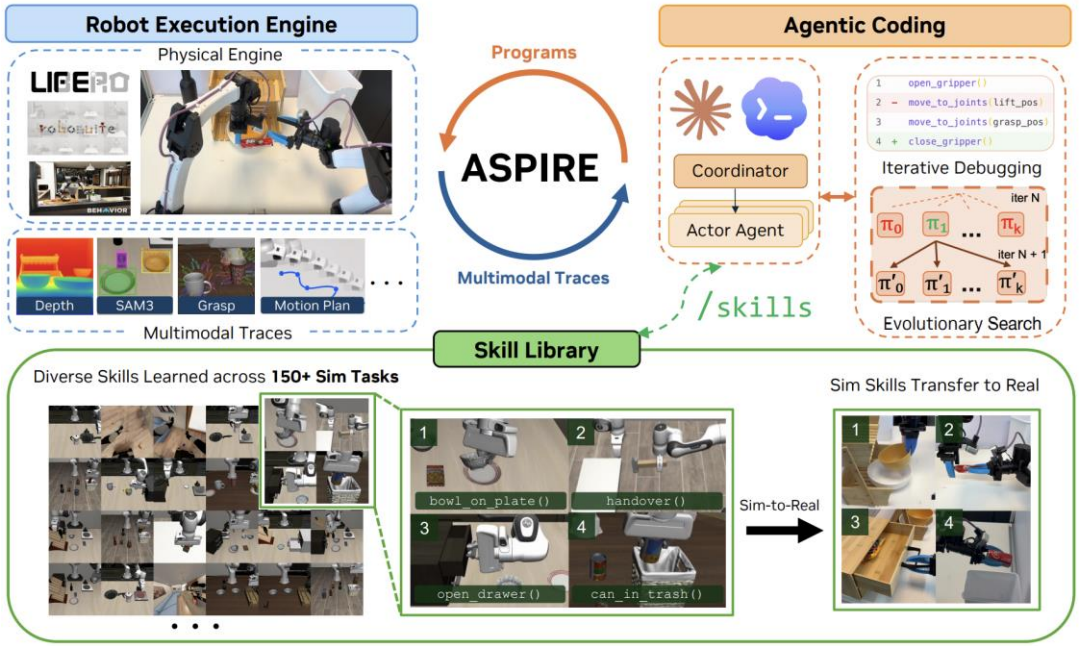
➡ **Self-improving continual learning robotic system인 ASPIRE 제안**

- Code-as-policy 패러다임

  → robot control program을 자동적으로 작성하고 refine하면서, 경험을 reusable한 skill library로 축적함
- Open-ended learning loop으로 실행

  → agent가 스스로 어떻게 실행 trace를 검사할지, 실패를 진단할지 등을 결정

Method



3개 요소가 open-ended learning loop을 구성

Coordinator-actor 구조 채택

- 🧠 **Coordinator**
  - 공유된 skill library를 관리
  - actor coding agent를 개별 task로 dispatch함
- 🤖 **Actor**
  - robot execution engine 내에서 로봇 프로그램을 쓰고, 실행하고, 진단하고, 수정함
  - 모든 chat history나 raw rollout trajectory를 교환하지 X
    transferable한 경험이 skill library로 distill됨

    → 각 actor의 context window가 현재 task에 집중 가능

Method: Robot Execution Engine

기존의 evidence 제공 방식: 수작업으로 선별된 scene-level summary / 미리 정의된 observations

→ **Trade off**: evidence 너무 적으면 **실패 primitive를 찾기 힘듦**, evidence 너무 많으면 **agent를 distract**할 수 있음

ASPIRE: **open-ended debugging environment**

- Per-primitive multimodal traces 기록
- Trace를 coding agent에 제공
- Closed-loop 검증을 위해 agent가 작성한 수정안 실행

Trace가 저장하는 것

- 호출된 API
- 입력 및 출력
- return status
- 관련된 multimodal evidence
  (RGB keyframes, overlays, grasp candidates, object poses, motion-planning results)

Method: Robot Execution Engine

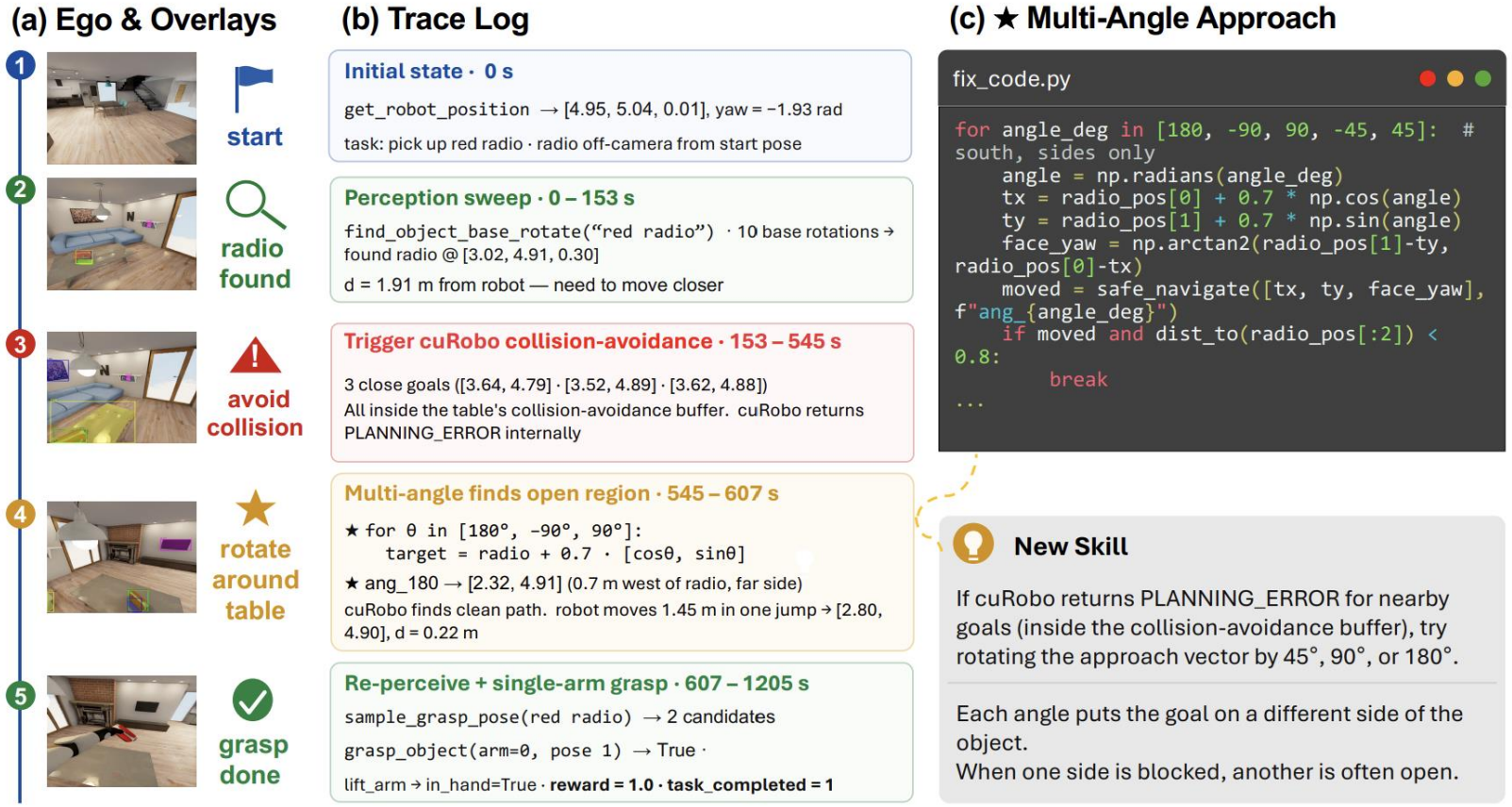


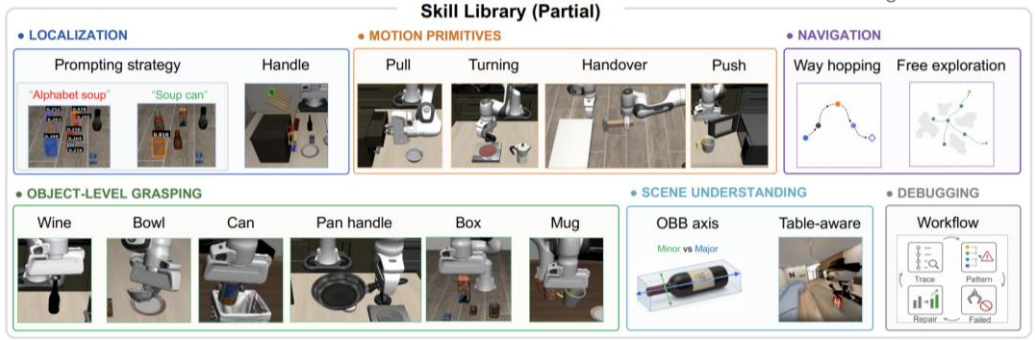
Figure 2: **Robot execution engine.** Trace-guided debugging on a BEHAVIOR-1K NAVIGATE-AND-PICK-UP-RADIO task. (a) Ego-view keyframes and overlays show the robot locating the radio but failing to approach it. (b) The primitive trace localizes the failure to repeated `PLANNING_ERROR`s: candidate navigation goals fall inside the table's collision-avoidance buffer. (c) The agent patches the program with a multi-angle approach routine, re-perceives the radio from a reachable side, and completes the grasp. The validated repair is admitted as a reusable *Multi-Angle Approach* skill.

Method: Skill Library

Skill Library에 **Heterogeneous repair knowledge** 저장

: localization heuristics, perception prompts, grasping constraints, navigation recovery strategies, motion primitives, scene-under standing routines, debugging workflows

Figure 3 Middle



Skill은 **compact in-context guidance**로 저장됨

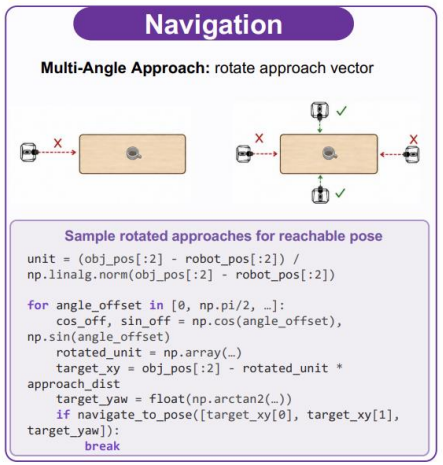
: 실패 신호, condition 적용 시점, repair 전략, (유용할 경우) 대표적인 code sketch 포함

→ Test 환경에서 매번 해결책을 추론을 통해 새로 찾아낼 필요 없이, 검증된 repairs 재사용 가능

⇒ 더 어려운 simulated task로 zero-shot transfer 가능

⇒ 다양한 로봇 형태로 일반화 되거나 실제 로봇으로 전이 가능한 메커니즘 제공

Figure 3 Top 中



Method: Skill Library

Actors (Coding agent)

- 실행 trace들에서 실패 진단
- 프로그램 patch
- 디버깅 환경에서 fix 검증
- 구조화된 findings 보고
  : failure mode, 검증된 fix, repair pattern

Coordinator

- findings 검토
  : API policy, 재사용 가능성
- validation 통과한 repair만 shared library에 등록



Figure 3: **Skill library.** Aspire stores validated, agent-discovered repair knowledge as reusable in-context skills rather than a fixed set of human-written primitives. Top: representative entries show learned skills about localization disambiguation, motion-primitive construction, and navigation recovery. Middle: the library grows across heterogeneous categories, including localization, navigation, motion primitives, object-level grasping, scene understanding, and debugging workflows. Bottom: selected skills discovered in sim are used as in-context guidance for real-robot programming, providing evidence that skills can transfer across embodiments. Additional skill examples and prompts are shown in Appendix A.

Method: Evolutionary Search

Algorithm 1 Evolutionary search over programs

Require: task τ, program P^0, sets $S_{\text{dbg}}, S_{\text{val}}$, skill library $\mathcal{L}$, agent M, budget (K, T), threshold θ
Notation: $\textsc{Execute}(P, S) = (r, Z)$ returns score r and trace bundle Z.

1: $(r^\star, Z^0) \leftarrow \textsc{Execute}(P^0, S_{\text{dbg}})$; $P^\star \leftarrow P^0$
2: $\mathcal{H} \leftarrow \{(P^0, r^\star, Z^0)\}$
3: **for** $i = 1, \ldots, T$ **do**
4: $\{P_i^k\}_{k=1}^K \leftarrow \textsc{ProposeRepairs}(M, \tau, \text{Top3}(\mathcal{H}), \mathcal{L}, \mathcal{H})$
5: **for** $k = 1, \ldots, K$ **do**
6: $(r_i^k, Z_i^k) \leftarrow \textsc{Execute}(P_i^k, S_{\text{dbg}})$
7: **end for**
8: $\mathcal{H} \leftarrow \mathcal{H} \cup \{(P_i^k, r_i^k, Z_i^k)\}_{k=1}^K$; $k^\star \leftarrow \arg\max_k r_i^k$
9: **if** $r_i^{k^\star} > r^\star$ **then**
10: $(P^\star, r^\star) \leftarrow (P_i^{k^\star}, r_i^{k^\star})$
11: **end if**
12: **if** $r^\star \geq \theta$ **then**
13: **break**
14: **end if**
15: **end for**
16: $(r_{\text{val}}, Z_{\text{val}}) \leftarrow \textsc{Execute}(P^\star, S_{\text{val}})$
17: $\mathcal{G} \leftarrow \textsc{ExtractValidatedPatterns}(\mathcal{H}, P^\star, r_{\text{val}}, Z_{\text{val}})$
18: **return** $(P^\star, r_{\text{val}}, \mathcal{G})$

line 1: 초기 프로그램 P^0를 디버깅 환경에서 실행 → score, trace 저장

line 4: agent가 새로운 후보 프로그램들을 만듦

성능 상위 3개 프로그램 H, skill library L, 후보들 실행 기록 H를 참고함

line 5-8: 생성한 후보들 실행

line 9-15: 가장 높은 score 받은 후보가 현재까지의 최적 프로그램보다 좋은 score 냈다면, 최적 프로그램 갱신

line 16: 가장 좋은 프로그램을 별도의 검증 환경에서 실행
→ 환경 변화에서도 효과가 이어지는지 확인

line 17: 지금까지의 탐색 이력, 최종 프로그램의 검증 결과를 바탕으로 재사용할 수 있는 해결 패턴 추출

Trace-guided debugging 자체만으로는 **local repair loop**에 빠질 수 있음

→ Evolutionary search 사용으로 다양한 repair 가정과 task 전략을 가지도록 **robot program의 exploration을 넓힘**

Experiments: Benchmarks and Baselines

Benchmarks

- LIBERO-Pro
- Robosuite
- BEHAVIOR-1K

For zero-shot transfer,

- LIBERO-Pro Long: 평가용
- LIBERO-90: skill 축적용

Baselines

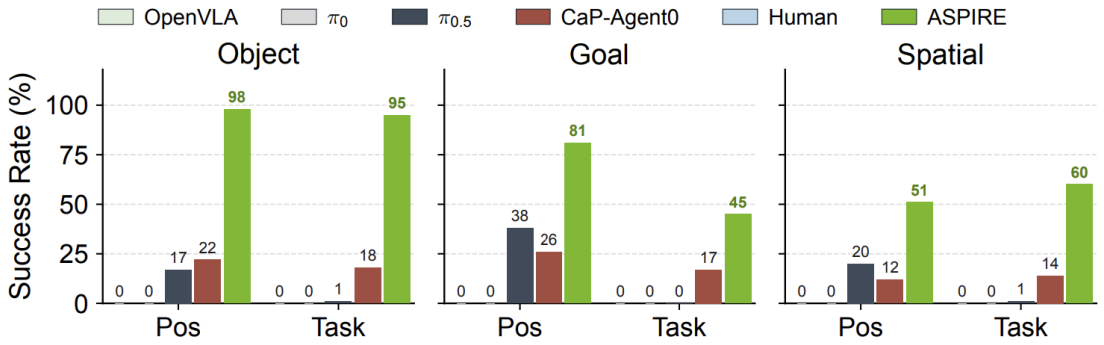
- Coding-agent: CaP-Agent0
- End-to-end VLA policy: OpenVLA, π_0, $\pi_{0.5}$

구분	OpenVLA, π_0, $\pi_{0.5}$	CaP-Agent0
분류	End-to-end VLA policy	Code-as-Policy coding agent
로봇 제어 방식	관측과 명령을 바탕으로 action을 예측	perception, control API를 조합한 프로그램을 생성/실행
실패 대응	이 비교에서 별도의 코드 수정 과정 없음	실행 결과를 보고 test-time에 코드 수정/재시도

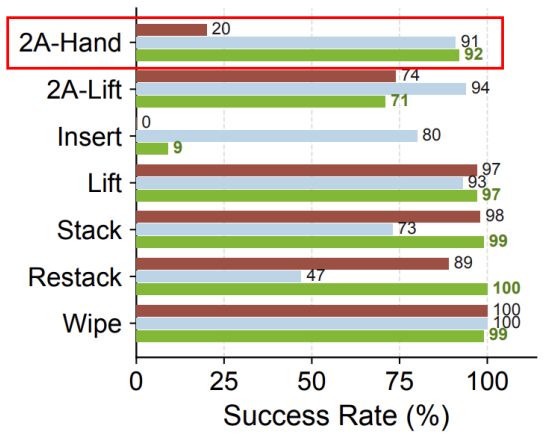
Experiments: Main Evaluation Results

★ Robosuite의 2A-Hand task(한 손에서 다른 손으로 물체 hand-over)에서 큰 성능 차이

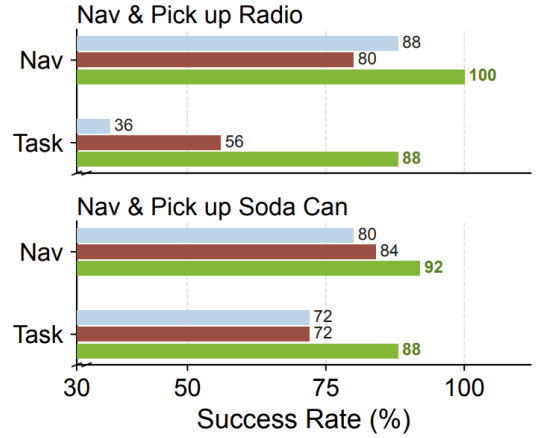
→ **실행 기록**을 통해 **실패 지점을 좁히고, 관련 코드를 수정**할 수 있는 구조 덕분으로 해석 가능



(a) **LIBERO-Pro** [Zhou et al., 2025]: macro-averaged success over 10 tasks × 50 held-out environment seeds per suite/perturbation. For each task, ASPIRE learns and collects skills on seeds 51–65 and evaluates one generated program on seeds 1–50.



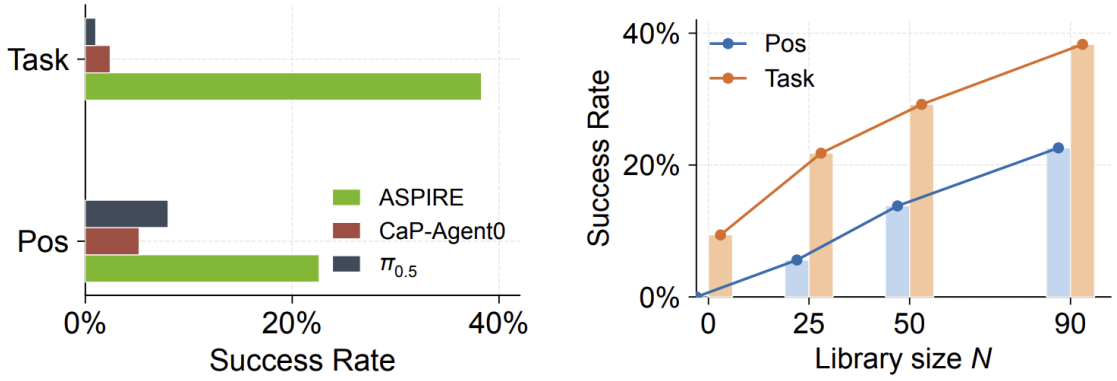
(b) **Robosuite** [Zhu et al., 2020]: success over 100 held-out trials per task. ASPIRE learns on seeds 101–125 and evaluates one generated program per task on seeds 1–100; 2A tasks are bimanual.



(c) **BEHAVIOR-1K** [Li et al., 2024]: long-horizon mobile manipulation on two household tasks. ASPIRE learns on seeds 26–35 and evaluates on seeds 1–25 with incremental block execution; Navigation and Task success are reported separately.

Figure 4: **Aspire improves over prior coding agents and end-to-end VLAs across three benchmark families.** (a) Short-horizon manipulation on LIBERO-Pro; (b) contact-rich manipulation on Robosuite; (c) long-horizon mobile manipulation on BEHAVIOR-1K. Aspire evaluates one generated program per task across held-out seeds, while CaP-Agent0 regenerates a separate program per seed with test-time reasoning and retries. Aspire improves performance across all benchmarks, with several results surpassing programs written by human experts. Detailed per-task results are in Appendix B and D.1.

Experiments: Zero-Shot Transfer to Unseen Tasks



(a) Full N=90 library zero-shot results versus baselines.

(b) Zero-shot success improves as skill-library size increases.

Figure 5: **Cross-task zero-shot transfer on LIBERO-Pro Long.** Skills accumulated on LIBERO-90 improve zero-shot performance on held-out long-horizon tasks. Figure (a) compares the full N=90 library with baselines. Figure (b) shows Pos/Task success as the size of the skill library increases. All success rates are macro-averaged over 10 tasks per axis. Per-task results are in Appendix C.

(a) LIBERO-90에서 skill library 축적 후 LIBERO-Pro Long에 적용하여 성능 비교

Success Rate 값은 높지 X △ 여러 단계가 연결된 처음 보는 작업을 추가 디버깅 없이 수행함

→ **짧은 작업에서 얻은 해결 경험**이 **새로운 장기 작업 중 일부를 실제로 해결**하는 데 사용됨

(b) LIBERO-90에서 다양한 task 개수로 skill library를 구성한 후 성능 비교

→ skill library 사이즈가 **커질수록** success rate도 **증가 =** 이전 작업에서 쌓는 **해결 경험이 다양**해질 수록, 처음 보는 장기 작업에서 **참고할 수 있는 스킬도** ⬆

Experiments: Real-Robot Skill Transfer Across Embodiments

Franka-based simulation → Bimanual YAM robot

Franka 기반 simulation에서 발견한 **Skill 3개**를 real-robot의 coding agent에게 **in-context guidance**로 제공

→ Skill을 제공하지 않은 경우와 비교

Skill은 실제 로봇용 완성 코드를 제공하는 것이 아니라, agent가 참고할 repair 지식을 제공함

→ 단순히 simulator 특화된 code를 외우는 대신, skill이 real-robot program synthesis를 guide함

☑ 세 task 모두 token 사용량은 줄었지만, success 향상 폭은 task마다 다름

Task	Output Tokens (M) ↓		Total Tokens (M) ↓		Success Rate ↑	
	w/o Skills	w/ Skills	w/o Skills	w/ Skills	w/o Skills	w/ Skills
Put bowl on plate	0.05	0.04	8.65	5.11	20/20	20/20
Lift soda can	0.18	0.03	61.94	6.58	13/20	19/20
Open/push drawer	1.33	0.36	334.917	81.67	0/20	11/20

Table 1: **Real-robot cross-embodiment skill transfer.** For each task, we compare real-robot debugging with and without retrieving a corresponding simulation-discovered skill from Aspire's library. Token counts are measured until the first successful real-robot program, or until the debugging budget is exhausted for runs that do not reach success. Success rate reports evaluation trials for the generated programs.

Limitations

아직 완전 자율 real-world 학습이 아님

Real-world 로봇에서 지속적으로 학습하려면 안정적인 성공 판별, 안전한 reset, 안전 모니터링, calibration 유지가 필요함

Frontier LLM에 의존

ASPIRE는 Claude Opus 4.6을 사용함—더 작거나 성능이 낮은 LLM으로도 동일한 debugging loop를 유지할 수 있는지는 검증되지 않음

Predefined primitive API의 제약

Perception, planning, control API가 debugging을 용이하게 하지만, 동시에 agent가 표현할 수 있는 행동의 범위를 그만큼 제한함

제공되지 않은 기능이 필요한 행동은 구현하기 어려움

불완전한 long-term 메모리 관리

Skill library가 커지면서 일부 skill이 오래되거나, 지나치게 task-specific하거나, 중복되거나, 새로운 task에 부적절해질 수 있음

더 견고한 retrieval, pruning, ranking, 재검증 메커니즘이 필요

+) 실제로 zero-shot transfer 실험에서
비단조적(non-monotonic) 성능 변화 보임
(Appendix C.1)

높은 계산 비용

Debugging과 Evolutionary Search에 task당 많은 LLM 호출과 robot rollout이 필요함—규모를 키우려면 추론 비용을 낮추거나 search 효율을 높여야 함

Idea & Discussion

어떤 방식이 중심이 될까?

- 대규모 데이터로 학습한 VLA
- 실행 과정에서 경험을 축적하는 에이전트 시스템

두 방법을 결합해서, VLA가 기본 행동 생성하고 상위 에이전트가 실패 분석해 보완하는 구조도 생각해볼 수 있음

확장 아이디어

1. Skill library 검토 메커니즘
   - 중복 제거
   - Skill 적용 시 성능 검토 → 제거 등 관리
2. Agent가 모은 skill library로 VLA fine-tuning하는 hybrid 접근법
3. Skill 선택 시 skill의 efficiency를 score에 반영: success rate이 동일하게 나왔을 때 더 효율적인 action 취할 수 있음

토의 아이디어

Skill 선택 기준
: 실패 신호 + Evolutionary search에서 success rate가 높은 후보 선택

→ 후보 skill들의 success rate이 동일하게 나왔을 때 선택 기준 정의 X

→ Skill의 efficiency를 score에 반영하면, 더 효율적인 skill을 선택할 수 있을 것 같음 (확장 아이디어 3)

"동점이 나왔을 때만 적용해야 할까? Success rate이 충분히 높은 경우에 근소한 차이로 비효율적인 skill이 채택되는 것이 맞을까?"

Thank You